

Kommandos: grep & sed

Susanne Schaller, MMSc - susanne.schaller@fh-hagenberg.at

Andreas Scheibenpflug, MSc -
andreas.scheibenpflug@fh-hagenberg.at



Einsatz regular expressions in der bash:

grep Kommando Familie

- Mit dem Kommando `grep` lassen sich Zeichenketten in Dateien oder Strings (=Variablen) finden, die auf die angegebene regular expression passen (matchen).
- Wenn eine Zeile den Ausdruck matcht, wird sie default mäßig auf der Standardausgabe ausgegeben.
- Wird der Ausdruck nicht gefunden, erfolgt keine Ausgabe. Der Returncode `$?` des `grep` Kommandos ist dann `1`.

Varianten von grep

Drei grep Varianten vorhanden:

- *grep*

Standardvariante mit **B**asic **R**egular **E**xpressions (BRE), lässt aber bestimmte regexs NICHT zu.

Beispiele:

Die Metazeichen `? + { } / ( )` können nicht direkt verwendet werden, sondern nur in Kombination mit einem Backslash: `\? \+ \{ \} \/ \( \)`

- *grep -E (extended grep)*

lässt **E**xtending **R**egular **E**xpressions (ERE) zu, Metazeichen werden ohne `\` akzeptiert

- *grep -F (fast grep)*

Stark vereinfachte Version, lässt nur Strings zu, keine regulären Ausdrücke

grep - Syntax

- Syntax:

grep [-Optionen] regex [dateien]

- Optionen:

- c (count) gibt nur die Anzahl passender Zeilen aus
- h (hide) unterdrückt die Ausgabe der Dateinamen
- i (ignore) keine Unterscheidung zwischen Groß - und Kleinschreibung
- l (list) gibt nur die Namen der Dateien aus, die passende Zeilen enthalten
- n (number) stellt jeder Ausgabezeile die Zeilennummer aus der entsprechenden Eingabedatei voraus.
- o (only) gibt jeden Match in einer eigenen Zeile aus
- s (supress) unterdrückt Fehlermeldungen über nicht existierende Dateien
- v (vice versa) gibt alle Zeilen aus, die nicht zu regex passen
- E extended regular expressions
- F fast grep

- Weitere Details: **man grep**

sed

- sed (stream editor) ist ein nichtinteraktiver Texteditor für die Verwendung in der Kommandozeile oder in Skripts zum Editieren von Textfiles
- Sehr umfangreiche Funktionalitäten
- Wichtigste Aufgabe: Suchen und Ersetzen von Zeichenketten in Textfiles oder Variablen
- Weitere Information siehe:
<http://www.tty1.net/sed-tutorium/index.html>

Suchen und Ersetzen von Text mit sed

- sed verwendet regular expressions zum Suchen von Text in Files

Syntax:

```
sed 'ADDRESSs/REGEXP/REPLACEMENT/FLAGS' filename  
sed 'PATTERNs/REGEXP/REPLACEMENT/FLAGS' filename
```

- ADDRESS und PATTERNs siehe nächste Folie
- s ist die Abkürzung für substitute (s)
- / ist Trennzeichen, man kann aber auch bei Bedarf ein anderes verwenden: z.B. # oder ? oder ;
- REGEX gibt das zu suchende Muster im Text vor
- REPLACEMENT ist die Zeichenkette, mit der die gefundene Zeichenkette ersetzt wird

Suchen und Ersetzen von Text mit sed

- Adressen und Muster (ADDRESSES und PATTERNS)
- Können den Suchbereich einschränken, in denen nach REGEX gesucht wird
 - Angabe einer einzelnen Zeile durch die Zeilennummer
 - Angabe eines Bereichs von Zeilen durch Angabe der Zeilennummern der Startzeile und der Endzeile (keine Angabe bedeutet, dass der ganze File durchsucht wird)
 - Suche nach bestimmten Muster (=PATTERN als regex) und verwende später die REGEX nur in Zeilen, die das PATTERN enthalten

Suchen und Ersetzen von Text mit sed

- FLAGS können wie folgt angegeben werden:
 - g** Ersetze alle gefunden Textpassagen, die zu REGEX passen, mit REPLACEMENT (ohne g wird immer nur die erste in einer Zeile ersetzt)
 - n** Kann beliebige Zahl sein, ersetzt n-tes Vorkommen von REGEX mit REPLACEMENT.
 - p** Falls eine Ersetzung durchgeführt wurde, wird die geänderte Zeile ausgegeben
 - i** verwende REGEX ohne Rücksicht auf Groß- und Kleinschreibung

sed - Suchen und Ersetzen: Beispiele

Folgender File `mydata` ist gegeben:

```
# Instruction Guides
1. Linux Sysadmin, Linux Scripting etc.
2. Databases - Oracle, mySQL etc.
3. Security (Firewall, Network, Online Security etc)
4. Storage in Linux
5. Productivity (too many technologies to explore, not
much time available)
# Additional FAQs
6. Windows - Sysadmin, reboot etc.
```

Beispiel 1: Ersetze das Wort Linux in Linux-Unix

```
sed -r 's/Linux/Linux-Unix/' mydata
```

regex

replacement

Ersetzung erfolgt nur
einmal

```
# Instruction Guides
1. Linux-Unix Sysadmin, Linux Scripting etc.
2. Databases - Oracle, mySQL etc.
3. Security (Firewall, Network, Online Security etc)
4. Storage in Linux
5. Productivity (Too many technologies to explore, not
much time available)
# Additional FAQs
6. Windows - Sysadmin, reboot etc.
```

sed - Suchen und Ersetzen: Beispiele

- Grundsätzlich bearbeitet sed den angegebenen File und gibt die Änderungen aus.
- Die Ersetzungen werden nicht direkt in den gegebenen File geschrieben, sondern nur am Bildschirm ausgegeben.
- Erst durch Ausgabe-Umlenkung kann man den geänderten File in einem neuen File speichern
- Beispiel:

```
sed -r 's/Linux/Linux-Unix/g' mydata > changedmydata
```
- Der Originalfile wird nicht direkt überschrieben, der geänderte File kann durch Ausgabe-Umlenkung gespeichert werden!

sed - Suchen und Ersetzen: Beispiele

Beispiel 2: Ersetze alle Vorkommen des Wortes Linux in Linux-Unix

```
sed -r 's/Linux/Linux-Unix/g' mydata
```

Instruction Guides

1. **Linux-Unix** Sysadmin, **Linux-Unix** Scripting etc.
2. Databases - Oracle, MySQL etc.
3. Security (Firewall, Network, Online Security etc)
4. Storage in **Linux-Unix**
5. Productivity (Too many technologies to explore, not much time available)

Additional FAQs

6. Windows - Sysadmin, reboot etc.

Beispiel 3: Ersetze das 2.Vorkommen des Wortes Linux in Linux-Unix

```
sed -r 's/Linux/Linux-Unix/2' mydata
```

Instruction Guides

1. Linux Sysadmin, **Linux-Unix** Scripting etc.
2. Databases - Oracle, MySQL etc.
3. Security (Firewall, Network, Online Security etc)
4. Storage in Linux
5. Productivity (Too many technologies to explore, not much time available)

Additional FAQs

6. Windows - Sysadmin, reboot etc.

sed - Suchen und Ersetzen: Beispiele

Beispiel 4: Suche alle Zeilen, die einen Bindestrich enthalten und entferne alle Zeichen nach dem Bindestrich bis zum Zeilenende (PATTERN)

```
sed -r '/-/s/-.*//g' mydata
```

Pattern

```
sed -r '/ich/s/€1500/€5000/g' Gehalt.dat
```

Pattern

s - Command

Instruction Guides

1. Linux Sysadmin, Linux Scripting etc.
2. Databases
3. Security (Firewall, Network, Online Security etc)
4. Storage in Linux
5. Productivity (Too many technologies to explore, not much time available)

Additional FAQs

6. Windows

Beispiel 5: Suche im Zeilenbereich 1-4 nach Linux und ersetze Linux durch LINUX

```
sed -r '1,4 s/Linux/LINUX/g' mydata
```

Addresses

s - Command

Instruction Guides

1. **LINUX** Sysadmin, **LINUX** Scripting etc.
2. Databases - Oracle, MySQL etc.
3. Security (Firewall, Network, Online Security etc)
4. Storage in Linux
5. Productivity (Too many technologies to explore, not much time available)

Additional FAQs

6. Windows - Sysadmin, reboot etc.

sed - Suchen und Ersetzen : Beispiele

Beispiel 5: Entferne alle HTML-Tags aus einem HTML File

```
sed -r 's/<[^>]*>//g' myhtml | sed -r '/^$/d'
```

Erklärung:

1. Löschen alle HTML Tags
2. Lösche alle entstandenen Leerzeilen

```
<!DOCTYPE html>
<html>
<body>

<h1>My First Heading</h1>

<p>My first paragraph.</p>

</body>
</html>
```

Ausgabe:

```
My First Heading
My first paragraph.
```

Gierige und genügsame „regexs“

Normalerweise sind alle regulären Ausdrücke, die Ausdrücke wie `. *` enthalten, "gierig" (greedy). Sie holen sich so viele Zeichen wie möglich!

```
sed -r 's/<.*>/ /g' myhtml
```

Dieser reguläre Ausdruck `<.*>` würde den gesamten File matchen, da er das letzte Auftreten des Zeichens `>` suchen würde.

Anschließend wären nur mehr Leerzeilen im File!

```
<!DOCTYPE html>
<html>
<body>

<h1>My First Heading</h1>

<p>My first paragraph.</p>

</body>
</html>
```

Gierige und genügsame „regexs“

- Abhilfe: Um diese gierige regex genügsamer zu machen, gibt's folgende Möglichkeit.

```
sed -r 's/<[^>]*>///g' myhtml
```

Erklärung:

Der Ausdruck `<.*>` wird entschärft:

Die schließende spitze Klammer wird beim Matchen auf eine beliebige Zeichenfolge ausgeschlossen und kann daher nicht gefunden werden.

Das erste Auftreten des Zeichens „>“ beendet die Suche!

sed – Zeilen löschen: Beispiele

Beispiel 6: Entferne alle Leerzeilen aus einem Dokument

```
sed -r '/^$/d' mydocument
```

Beispiel 7: Entferne die erste Zeile eines Dokuments bzw. lösche den Zeilenbereich 7 bis 9.

```
sed -r '1d' mydocument
```

```
sed -r '7,9d' filename.txt
```

sed für Variablenbearbeitung

- Eine Variable kann sed nicht direkt bearbeiten
- Aber: Kommandosubstitution mit Pipe verwenden
- Beispiel: Umwandlung Umlaute in Deutsch: ä zu ae, ö zu oe, ü zu ue

```
A="Löschfahrzeug"
```

```
ANEU=`echo $A | sed -r 's/ö/oe/g'`
```

```
echo $ANEU
```

```
Loeschfahrzeug
```

```
A=$ANEU
```